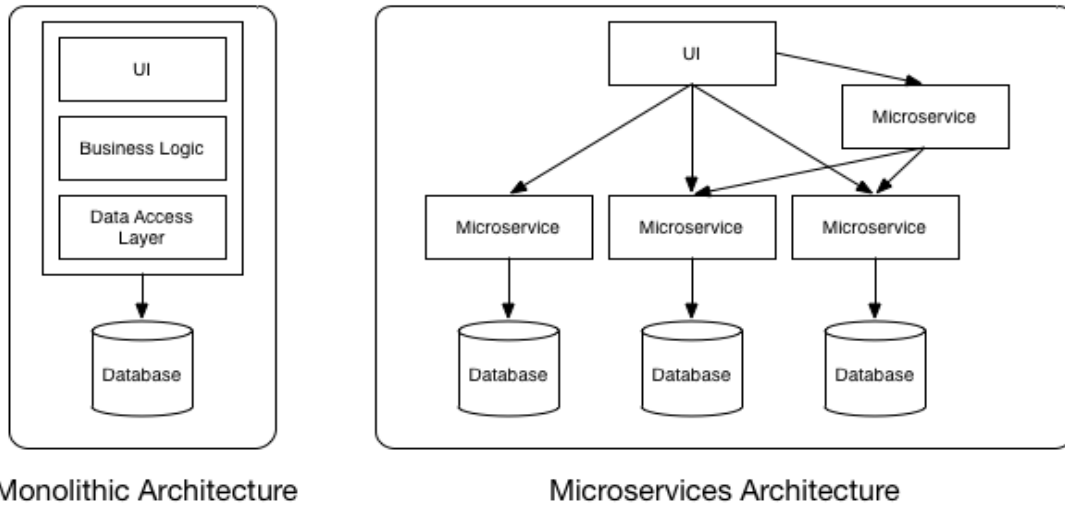


Monolithic Architecture



What is monolithic architectural?

Monolithic means an **architectural style /pattern**

A monolithic **application is built as a single deployment unit**. (A single jar file) and they run as a **single process**.

In a monolithic web application, a typical stack could include:

1. **Frontend:** HTML, CSS, and JavaScript
2. **Backend:** A single server or framework (Springboot/Django) handling business logic
3. **Database:** A single relational database containing all application data

When to Use Monolithic Architecture:

Monolithic architecture is often chosen for Small to medium-sized applications

Monolithic Application Characteristics:

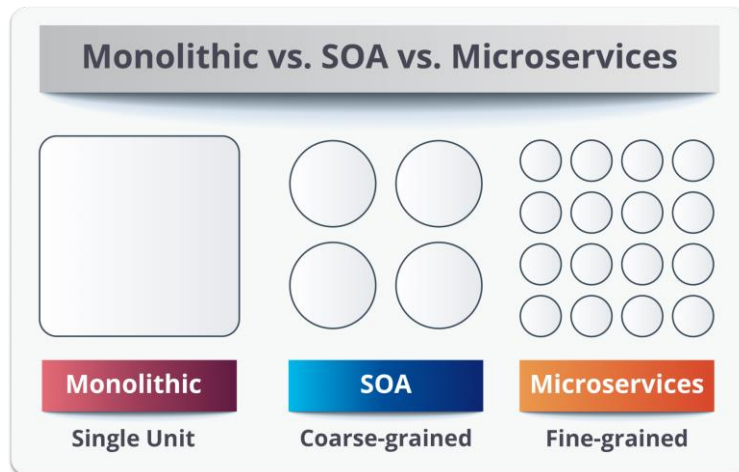
1. **Single Codebase:** All the components (user interface, business logic, data access) are written and maintained in one codebase or repository e.g. github.
2. **Tightly Coupled Components:** Different parts of the application are tightly interconnected, which means **changing one component often affects others**.
3. **Single Deployment Unit:** The entire application is deployed as one unit, which **means updating or modifying one part requires redeploying the entire application**.
4. **Shared Resources:** Typically, a monolithic application shares a **single database**.

5. **Easy to scale:** The App ran **multiple instances** of the application **behind a load balancer** **distributing traffic across them**
6. **Easy to Test:** Monolithic applications can be **relatively** easy to test, especially with **end-to-end tests**. You can launch the entire application, interact with it through its API or UI (using tools like Selenium), and verify the expected behavior. This is because all components are tightly integrated within a single unit.
7. **Less Network Overhead:** As everything runs in a single process, there is no network communication required between services, which can reduce latency.
8. **Single Environment:** Monolithic applications generally require only one server or a few servers, leading to less complexity in terms of operations.

Disadvantages of Monolithic Applications:

1. **Tightly Coupled:** All modules are **developed** and **deployed** as a single unit. ie a single executable jar file
2. **Difficult Agile Practices:** Tight coupling hinders frequent deployments, making **CI/CD** tough.
3. **Long Deployment Time:** Entire app needs redeployment for any small change,
 - a) delaying updates (Production hotfixes take longer)
 - b) new features take a long time to be released.
4. **Long Development Cycle:** Close coordination among teams slows development and reliability.
5. **Scaling Challenges:** Individual modules are difficult to scale. Also conflicting resource (e.g. one module needs more CPU, another needs more memory need)s among modules make scaling difficult.
6. **Single point of Failure:** One failing module can crash the entire system.
7. **Reliability and Availability Issues:** due to high coupling between modules
8. **Reduced Agility:** CI/CD practices are challenging due to tight coupling and long development cycles.
9. **Slow Startup:** Large app size increases web server and IDE startup times.
10. **Reduced Developer Productivity: Overloaded IDEs** slow down development.
11. **High Dependency on Key Developers:** The entire codebase is understood by only a few key developers.
12. **Tech Constraints:** Unable to adopt newer technologies for better performance due to their close relationship with one technology

As a solution to some of the **limitations of the monolithic application architecture**, **Service Oriented Architecture (SOA)** and Enterprise Service Bus (ESB) emerged.



Additional Information:

Creator: The term "Microservices" was popularized by **Martin Fowler** and **James Lewis** in 2014 through their article "Microservices – a definition of this new architectural term."

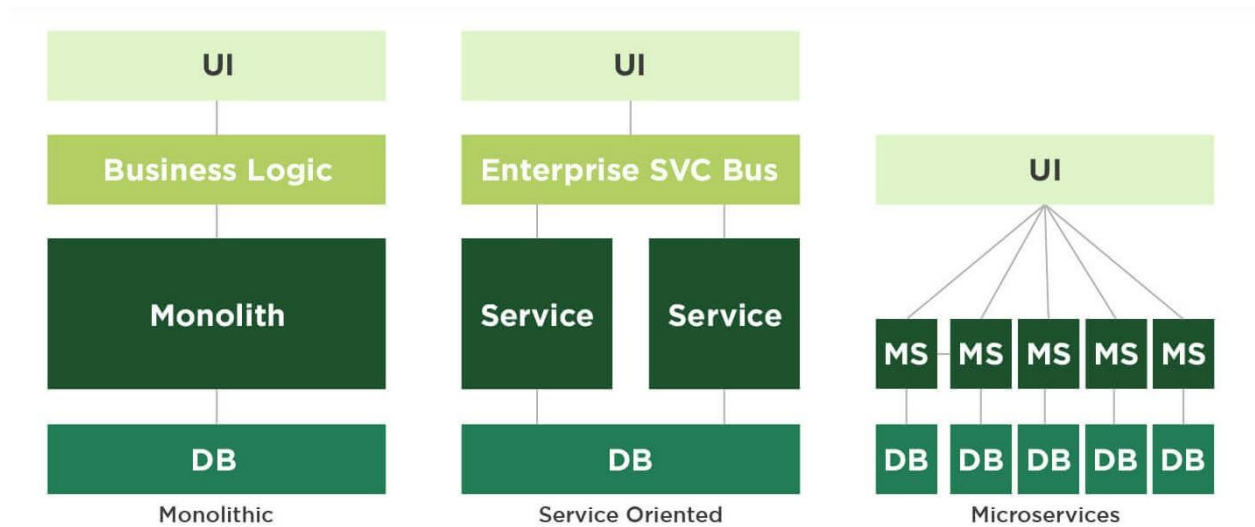
Start: Microservices began gaining traction around **2010**, as **cloud computing and DevOps practices became mainstream**

Creator: SOA does not have a single, definitive creator. It evolved as a design paradigm in the early 2000s, influenced by industry leaders like **IBM**, **Microsoft**, and **Oracle**.

Start: The concept gained prominence in the **early 2000s**, though its principles can be traced back to distributed computing and CORBA (Common Object Request Broker Architecture) from the 1990s.



Microservices



Introduction: -

Microservices is an architecture style in which a single application is composed of many micro or small **services**. These small services are loosely coupled and independently deployable

These are small services which

1. runs in its **own process** and
2. **Communicating with lightweight mechanisms** eg: http
3. **Independently deployable**

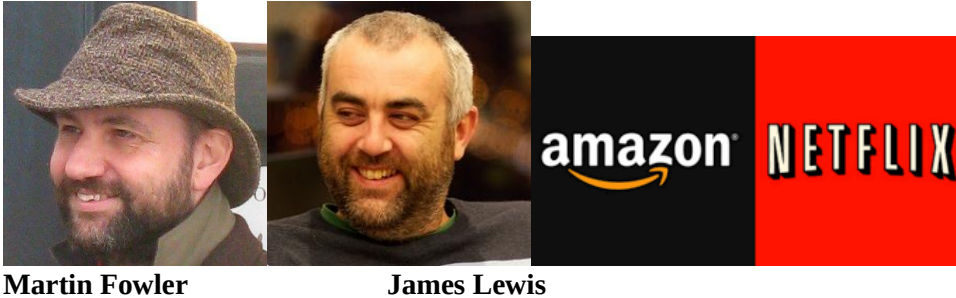
Fowler is credited with promoting the idea of breaking down services in a service-oriented architecture (SOA) into microservices. In a 2014 article, he stated:

"The microservice architectural style is an approach to developing

1. a **single application** as a suite of small services,
2. each running in its **own process** and
3. **Communicating with lightweight mechanisms**, often an HTTP resource API.
4. These services are built **around business capabilities** and
5. **Independently deployable** by fully **automated deployment machinery**.

Note:-

- Microservices don't have a single creator but evolved over time.
- **Martin Fowler and James Lewis** popularized the term in 2014 through their ThoughtWorks article.
- Companies like **Amazon and Netflix** were early adopters, using microservices to solve scalability and reliability challenges, bringing the concept into the mainstream.



Martin Fowler

James Lewis

Capabilities, Features, and Benefits of Microservices Architecture

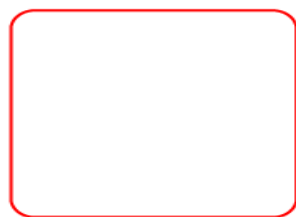
1. **Decentralized Development:** Each service operates independently with its own database, enabling decentralized development and deployment.
2. **Loosely Coupled Services:** Services are connected via lightweight APIs, allowing independent deployment and operation.
3. **Frequent Deployment and Continuous Delivery:** Smaller, modular services enable faster deployment and continuous delivery.
4. **High Scalability:** Services can scale independently, optimizing resource use and reducing costs.
5. **Ease of Adding Features:** Modular design allows for easy integration of new features without impacting other services.
6. **Technology Flexibility (Polyglot):** Different technologies can be used for each service, avoiding vendor lock-in.
7. **Code Reusability:** APIs expose functionality that can be reused across different services or applications, enhancing development efficiency.
8. **Fault Isolation:** Failures in one service don't affect others, improving overall system reliability.

Drawbacks of Microservices Architecture:

1. **Complex Service Management:** Managing a growing number of services can become complex as each service adheres to the Single Responsibility Principle (SRP).
2. **Database and Transaction Management:** Each service often has its own database, leading to challenges with consistency, management, and handling distributed transactions.
3. **Testing Challenges:** Requires multiple levels of testing:
 - a. Unit Testing for each service

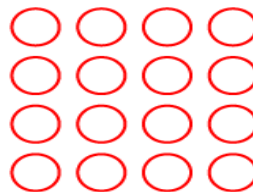
- b. Integration Testing for interactions
 - c. End-to-End Testing for system behavior
- 4. **Versioning Issues:** Updates can break backward compatibility, requiring careful coordination and testing.
- 5. **Deployment Complexity:** Automation (e.g., CI/CD) is necessary for managing the deployment of many services; manual deployment is unmanageable.
- 6. **Centralized Logging:** Aggregating logs from multiple services adds overhead in setup and maintenance.
- 7. **Monitoring and Observability:** Requires centralized tools to monitor service health and quickly diagnose issues.
- 8. **Debugging Challenges:** Traditional debugging methods don't work across multiple services, requiring specialized tools and practices.
- 9. **Connectivity and Service Discovery:** Managing service discovery and communication adds operational complexity.
- 10. **Network Communication Overhead:** Increased latency and the need for high-bandwidth infrastructure due to inter-service communication over the network.
- 11. **Operational and Infrastructure Overhead:** Requires significant effort and investment in managing services, CI/CD pipelines, monitoring tools, and infrastructure

Monolithic Vs Microservices



Monolithic

Single Unit



Microservices

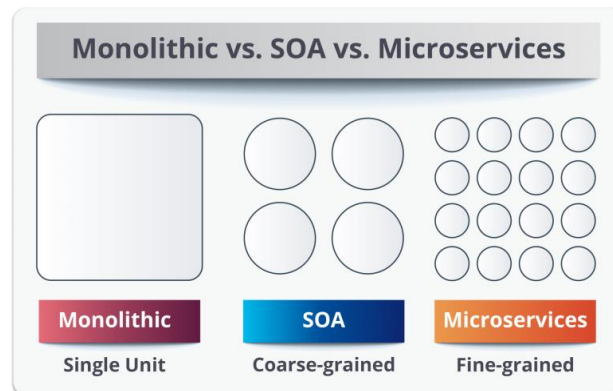
Fine-grained

Difference between Monolithic and Microservice

Feature	Monolithic	Microservices
Definition	A single unified application where all modules are tightly coupled .	A collection of small, loosely coupled services, each focusing on a specific business capability .
Architecture	Multi-tiered but tightly coupled ; all layers reside in a single deployment unit .	Distributed and loosely coupled ; each service is independently deployable and communicates via APIs.
Scalability	Vertical Scaling (Scaling Up)(adding more resources, such as CPU, RAM, or storage.to a single server).	Horizontal Scaling (Scaling Out)(adding more servers/services and distributing traffic among them using a load balancer.).
Deployment	Entire application is deployed as a single unit.	Individual services can be deployed independently.
Development	One large codebase, often requiring coordination among large teams.	Independent teams can work on separate services simultaneously.
Fault Isolation	A failure in one module can affect the entire application.	Failure in one service usually does not impact others.
Technology Stack	Typically uses a single technology stack for the entire application.	Each service can use a different technology stack.
Database	Single database for all services.	Each service has its own separate database.
Maintenance	Harder to maintain as the codebase grows.	Easier to maintain due to smaller, focused services.
Performance	Better for small applications with less complexity .	May have overhead due to inter-service communication.
Testing	Easier to test as a single unit.	Requires more complex testing for inter-service communication.
Team Organization	Centralized team structure. (Single large team works on whole application)	Decentralized teams with service-specific expertise. (Separate small team work on each Service which are more focused.)
Use Case	Suitable for small to medium-sized applications with simple requirements.	Ideal for large, complex, and evolving applications.

What is SOA?

Service-Oriented Architecture (SOA) is **an architectural style** that supports service-orientation. Service-orientation is **a way of thinking in terms of services**



What are the similarities between SOA and MSA

At first glance, the two approaches ie SOA and MSA sound very similar, and in some ways, they are.

- 1) Both SOA and MSA **are service-based architectures**
- 2) Both break large, complex applications into **small, flexible components that are easier to work with.**
- 3) Both involve **cloud or hybrid cloud environments for** agile application development and deployment,
- 4) They share **platform independence, language independence, and location independence**

Difference between SOA and Microservices

Aspect	SOA	Microservices
Service Size	Coarse-grained (Big, Broad Services) (They combine multiple related tasks into one unit. Example: A single service managing all operations for "Order Management.")	Fine-grained (Small, Focused Services): (Each service performs one task and does it well. Example: Separate services for "Order Creation," "Order Tracking," and "Order Payment.")
Communication	Uses a central system (ESB). enterprise service bus	Direct communication (REST, HTTP , etc.).
Data Storage	Services share the same database .	Each service has its own database .
Coupling	More dependent on other.	Independent and loosely connected.
Technology	Single tech stack.	Mixed tech stack (polyglot).
Scalability	Harder to scale parts separately.	Easy to scale individual services.
Deployment	Slower and centralized.	Faster and independent.
Failures	One failure can affect the whole system.	Failures are isolated to one service.
Best For	Large, stable enterprise apps.	Modern, flexible, and scalable apps.

Key Principles of Microservices Architecture

1. Single Responsibility Principle (SRP):

- **Focus:** Each microservice should have a single, well-defined responsibility
- **Benefit:** This promotes modularity, making services easier to develop, maintain, and scale independently. Changes to one service have minimal impact on others.

2. Loose Coupling:

- **Focus:** Minimize dependencies between services.
- **Benefit:** Services can evolve independently, reducing the risk of cascading failures and enabling faster development cycles.

3. Independent Deployments:

- **Focus:** Each service should be independently deployable. (Microservices can be **developed, deployed, and scaled** independently)
- **Benefit:** Allows for faster and more frequent releases, improving agility and reducing time-to-market.

4. Business Capabilities:

- **Focus:** Align microservices with business capabilities or domains.
- **Benefit:** Improves organizational alignment and makes it easier to understand and manage the system.

5. Decentralized Governance:

- **Focus:** Avoid centralized control over technology choices.
- **Benefit:** Empowers development teams to choose the best technologies for their specific needs, fostering innovation.

6. API-First Approach:

- **Focus:** Define clear and well-documented APIs for each service.
- **Benefit:** Enables seamless communication and integration between services, both within and outside the organization.

7. Data Ownership:

- **Focus:** Each service owns its own data.
- **Benefit:** Promotes data independence and allows services to evolve their data models independently.

8. Fault Tolerance:

- **Focus:** Design services to be resilient to failures.

- **Benefit:** Ensures that the system can continue to function even if individual services fail.

9. Observability:

- **Focus:** Build in mechanisms for monitoring and logging service performance.
- **Benefit:** Enables proactive identification and resolution of issues, improving system stability and reliability.

10. Continuous Delivery:

- **Focus:** Automate the build, test, and deployment processes for each service.
- **Benefit:** Services are small, enabling faster development, testing, and deployment cycles Reduces manual effort, accelerates delivery cycles, and improves software quality.

